

Módulo 3 · Testing, Debugging y Documentación con IA

IA en el Pipeline: Tests, Fallos, Seguridad y Buenas Prácticas

Ejercicio de Clase · Ejercicio 2 de 2 · Bloques 4.4 — 4.5 — 4.6 — 4.7

Nombre del alumno/a	Angel Echenique
Fecha	26/03/2026
🔗 Enlace GitHub	

📌 **Instrucciones:** Responde con lo que vimos en clase. Puedes consultar tus apuntes. En el Bloque 5 deberás investigar por tu cuenta y citar la fuente. ¡Solo demostrar que entendiste, no hay que escribir código!

BLOQUE 1 — ¿Qué puede hacer la IA en el pipeline? (4.4)

Vimos que la IA no es solo un complemento, sino un agente activo. Para cada capacidad, escribe para qué sirve y un ejemplo concreto de cómo lo haría la IA en un pipeline real:

Capacidad de la IA	¿Para qué sirve?	Ejemplo concreto en un pipeline real
Generar pruebas unitarias y de integración	Para aumentar la cobertura de tests sin que nosotros tengamos que escribirlos todos a mano. Por otro lado la IA detecta rutas lógicas sin probar y propone tests que cubren casos normales y edge	Tras un git push, el pipeline lanza un paso de IA que analiza el diff del último commit y genera nuevos tests en pytest para las funciones modificadas. Si ve que una función no tiene tests para valores nulos, añade uno automáticamente.
Analizar logs y explicar fallos	Para no perder 20 minutos leyendo logs crípticos. La IA resume qué ha pasado y cuál es la causa raíz.	Cuando un test falla, el paso de IA recibe el log completo del job, lo resume y devuelve un mensaje tipo: "El fallo se debe a que dividir() no lanza ValueError al dividir por cero; el cambio está en calculadora.py línea 12".

Detectar vulnerabilidades (OWASP Top 10)	Para encontrar riesgos de seguridad antes de desplegar: dependencias vulnerables, uso inseguro de funciones, exposición de datos, etc.	La IA analiza el requirements.txt y el código; detecta que se usa requests==2.25.0 con una CVE conocida y marca el build como riesgo alto, sugiriendo actualizar a una versión segura.
Generar documentación técnica y reportes	Para que el equipo tenga una visión clara sin leer todos los pasos del pipeline.	Al final del pipeline, la IA genera un informe con: número de tests, cobertura, vulnerabilidades, tiempo total y recomendaciones, y lo adjunta como artefacto o lo envía por email al equipo.
Proponer refactorizaciones	Para reducir deuda técnica y mejorar la calidad del código de forma continua.	La IA detecta una función de 200 líneas con código duplicado y sugiere extraer funciones más pequeñas, renombrar variables y separar responsabilidades, indicando archivos y líneas concretas.
Auditar seguridad del código	Para revisar patrones inseguros más allá de un simple escáner de dependencias: credenciales hardcodeadas, uso de eval, permisos excesivos, etc.	La IA analiza los workflows de GitHub Actions y detecta que un token tiene permisos write-all. Sugiere limitarlo a read y solo a los repos necesarios.

1.2 — Un pipeline típico con IA: ¿en qué orden actúa?

Vimos que un pipeline típico con GitHub Actions + IA tiene etapas específicas. Ordénalas correctamente del 1 al 7 escribiendo el número en la columna izquierda:

Orden	Etapas del pipeline con IA
5	IA analiza resultados del pipeline
2	Instalación de dependencias
6	IA sugiere mejoras o refactorizaciones
1	Checkout del repositorio
3	Ejecución de tests
7	IA genera reporte técnico automático
4	Análisis de seguridad

BLOQUE 2 — IA generando tests dentro del pipeline (4.5)

En clase vimos que la IA puede generar tests automáticamente en momentos clave del ciclo de desarrollo. Completa la tabla:

Momento clave en que actúa la IA	¿Qué genera exactamente?	¿Por qué es útil en ese momento?
Al crear un Pull Request	Tests unitarios para las funciones modificadas y tests de integración para los endpoints afectados.	Porque detecta errores antes de fusionar el código. Evita que llegue código roto a main.
Al detectar baja cobertura	Tests adicionales para cubrir ramas lógicas sin probar: valores nulos, casos límite, errores esperados.	Porque aumenta la calidad del proyecto sin que el equipo tenga que escribir esos tests manualmente.
Antes de un despliegue	Tests de integración y tests de regresión para asegurar que el sistema completo funciona.	Porque evita que un cambio aparentemente pequeño rompa producción.
Tras un fallo en el pipeline	Un test de regresión que reproduce exactamente el fallo detectado.	Porque garantiza que ese error no vuelva a ocurrir en el futuro.

2.2 — Tipos de tests que puede generar la IA

Clasifica cada ejemplo de test en su tipo: U (Unitaria), I (Integración) o R (Regresión), y explica brevemente por qué:

U/I/R	Ejemplo de test generado por la IA	¿Por qué es de ese tipo?
U	Test que verifica que <code>validar_email()</code> devuelve False con un email vacío	Prueba una sola función aislada, sin dependencias externas.
R	Test que reproduce el error detectado en el commit anterior (<code>ZeroDivisionError</code>)	Su objetivo es evitar que un error ya corregido vuelva a aparecer.
I	Test que simula el flujo completo registro → login → perfil → logout	Comprueba varios módulos interactuando entre sí.
U/I/R	Ejemplo de test generado por la IA	¿Por qué es de ese tipo?
U	Test que verifica <code>calcular_total()</code> con <code>precio = 0</code> y <code>cantidad = 1</code>	Solo evalúa una función concreta con un caso edge.
I	Test que comprueba que el endpoint POST <code>/pagos</code> devuelve 201 con datos válidos	Involucra API, servidor, validaciones y posiblemente base de datos.
R	Test que evita que vuelva a ocurrir un error de autenticación ya corregido	Su propósito es asegurar que un bug pasado no reaparezca.

2.3 — Prompts típicos dentro del pipeline

En clase vimos ejemplos de prompts que el pipeline puede incluir para pedirle tests a la IA. Escribe el prompt que usarías para cada situación:

Situación	Prompt que escribirías para la IA (dentro del pipeline)
Generar tests unitarios para el código del último commit, en pytest, con casos edge	Generame tests unitarios en pytest para el código modificado en el último commit. Incluye casos edge, valores nulos y errores esperados. No repitas tests existentes.
Generar tests de integración para los endpoints modificados, con validación de HTTP y casos de error	Generame tests de integración para los endpoints afectados por este commit. Valida códigos HTTP, respuestas JSON, casos de error y permisos.
Generar tests de regresión que reproduzcan el fallo detectado en este pipeline	Genera un test de regresión que reproduzca el fallo detectado en este pipeline. Usa los logs y el stacktrace para replicar exactamente el error.

BLOQUE 3 — IA explicando fallos y detectando vulnerabilidades (4.6)

Vimos dos grandes capacidades de la IA dentro del pipeline: explicar fallos y detectar vulnerabilidades de seguridad.

3.1 — IA explicando fallos del pipeline

La IA puede explicar la causa raíz de un fallo, no solo el mensaje de error. Completa la tabla con qué puede explicar la IA en cada caso:

Tipo de fallo en el pipeline	¿Qué explica y qué sugiere la IA?	
Un test falla con AssertionError	Me dice que valor esperaba el test, qué valor obtuvo realmente y qué parte del código causó la diferencia.	Revisar la lógica de la función, añadir validaciones o actualizar el test si el comportamiento esperado cambió.
El pipeline se bloquea por una dependencia rota	Identifica qué librería falló, si la versión no existe, si hay un conflicto o si la dependencia tiene una vulnerabilidad conocida.	Actualizar la versión, fijar versiones en requirements.txt o regenerar el entorno.
El despliegue falla por un error de permisos	Señala qué permiso falta: acceso al registry, permisos de escritura, token insuficiente, etc.	Ajustar los permisos del token, usar un rol más adecuado o corregir la configuración del workflow.
Un endpoint devuelve 500 en las pruebas de integración	Resume el stacktrace, identifica la función interna que lanzó la excepción y relaciona el error con el commit reciente.	Manejar excepciones, validar entradas, revisar la lógica del endpoint o añadir tests adicionales.

3.2 — IA detectando vulnerabilidades en CI/CD

La IA actúa como analista de seguridad dentro del pipeline. Para cada tipo de vulnerabilidad, escribe qué detectaría la IA y qué solución sugeriría:

Tipo de vulnerabilidad	¿Cómo lo detecta la IA?	¿Qué solución sugiere?
Dependencia con versión vulnerable (ej: requests==2.25.0)	Comparando la versión con bases de datos de CVEs y analizando el requirements.txt.	Actualizar a una versión segura y fijar versiones estables.

Credenciales hardcodeadas en el código (ej: password = "admin123")	Reconoce patrones típicos de contraseñas, tokens o claves API dentro del código.	Mover las credenciales a variables de entorno o secretos del pipeline.
Uso de eval() con entrada del usuario	Identifica funciones peligrosas y analiza si la entrada proviene del usuario.	Reemplazar eval() por alternativas seguras o sanitizar la entrada.
Permisos excesivos en el archivo de workflow (write-all)	Revisa el YAML del pipeline y detecta permisos demasiado amplios.	Restringir permisos a solo lectura o a los recursos estrictamente necesarios.

3.3 — IA generando reportes automáticos

Vimos que la IA genera un reporte profesional al finalizar el pipeline. ¿Qué secciones incluye ese reporte? Escribelas y describe qué información aporta cada una:

Sección del reporte	¿Qué información aporta al equipo?
Resumen ejecutivo (métricas)	Incluye: número de tests, tiempo total, cobertura, riesgos detectados. Sirve para que el equipo entienda el estado del proyecto en 10 segundos.
Tests fallidos	Lista qué tests fallaron, por qué y cuál es la causa raíz. Ayuda a priorizar correcciones.
Cobertura de código	Indica qué porcentaje del código está cubierto por tests y qué archivos tienen baja cobertura. Permite decidir dónde generar más tests.
Vulnerabilidades detectadas	Enumera riesgos de seguridad, su criticidad y su ubicación. Permite actuar antes del despliegue.
Recomendaciones para próximos pasos	Sugerencias de la IA: refactorizar, mejorar rendimiento, añadir validaciones, etc.
Evolución de calidad (histórico)	Muestra si la calidad mejora o empeora con el tiempo: más tests, menos fallos, menos vulnerabilidades, etc.

BLOQUE 4 — Buenas prácticas y errores comunes (4.7)

Vimos en clase el flujo de trabajo profesional IA + Humano + Pipeline. Completa la tabla con las tareas clave de cada fase:

Fase	Responsable	Acción principal	Tareas clave (al menos 2)
1. Generación	IA	Crear la base	➤ Generar tests, documentación o código inicial. ➤ Analizar el código y proponer refactorizaciones.
2. Validación	Humano	Criterio y seguridad	➤ Ajustar o corregir lo que la IA propuso. ➤ Confirmar que los tests cubren los casos reales del negocio.
3. Ejecución	Pipeline	Garantía y despliegue	➤ Ejecutar tests, análisis de seguridad y build. ➤ Bloquear despliegues si hay fallos o vulnerabilidades. ➤ Generar reportes automáticos y notificar al equipo.

4.2 — Buena práctica o error común

Marca BP (Buena Práctica) o EC (Error Común). Para los errores, escribe cómo se corregiría:

BP / EC	Afirmación	Corrección (solo si es EC)
BP	Revisar siempre el código generado por la IA antes de integrarlo en el pipeline.	
EC	Confiar ciegamente en los tests generados por la IA y desplegarlos directamente a producción.	Revisar manualmente los tests, validar que cubren casos reales y ejecutar el pipeline completo antes de desplegar.
BP	Documentar en el código qué partes fueron generadas por la IA y cuándo se revisaron.	
EC	Permitir que la IA tenga acceso a secretos y credenciales del pipeline para funcionar mejor.	Mantener los secretos aislados y usar permisos mínimos.

		La IA no debe acceder a credenciales sensibles.
BP	Validar manualmente los mocks generados por la IA para comprobar que simulan bien la realidad.	
BP	Mantener revisión humana en las etapas críticas del pipeline aunque la IA haya dado el visto bueno.	
EC	No validar los resultados de la IA porque si los tests pasan, es que el código es correcto.	Revisar siempre la lógica, incluso si los tests pasan. Los tests pueden estar incompletos o mal diseñados.

4.3 — El checklist de validación humana

Vimos que antes de aceptar código generado o revisado por IA, el desarrollador debe pasar un checklist. Escribe al menos 3 preguntas para cada categoría:

Categoría del checklist	Preguntas que debes hacerte (mínimo 3 por categoría)
1. ¿Entiendo el código?	➤ ¿Puedo explicar qué hace cada función sin mirar la IA? ➤ ¿Tiene sentido la lógica y coincide con los requisitos? ➤ ¿Hay partes confusas, duplicadas o innecesarias?
2. ¿Cubre todos los casos?	➤ ¿Incluye casos edge (nulos, vacíos, límites)? ➤ ¿Cubre errores esperados y excepciones? ➤ ¿Prueba entradas válidas e inválidas?
Categoría del checklist	Preguntas que debes hacerte (mínimo 3 por categoría)
3. ¿Es seguro?	➤ ¿Hay datos sensibles expuestos? ➤ ¿Se validan las entradas del usuario? ➤ ¿Hay funciones peligrosas (eval, acceso directo a archivos, etc.)?
4. ¿Los tests son suficientes?	➤ ¿La cobertura es adecuada? ➤ ¿Hay tests de regresión para errores pasados? ➤ ¿Los tests de integración cubren el flujo real del usuario?

▣ **Caso real de síntesis:** Un equipo de desarrollo va a desplegar un módulo de pagos. La IA ha generado 60 tests de integración y el análisis de seguridad automatizado no ha dado alertas. El jefe de proyecto propone desplegar directamente esa tarde para cumplir el plazo.

4.4 — Análisis del caso real

¿Qué riesgos tiene esta decisión? ¿Qué harías tú de forma responsable? Apóyate en el checklist y en lo visto en el bloque:

¿Qué riesgos tiene esta decisión?

- los tests generados por IA pueden no cubrir casos reales del negocio.
- La IA puede haber omitido escenarios críticos (fraude, duplicidad de pagos, errores bancarios).
- El análisis de seguridad puede no detectar configuraciones inseguras o permisos incorrectos.
- Un fallo en pagos afecta dinero real: riesgo alto.

¿Qué harías tú de forma responsable?

- Revisar manualmente los tests más críticos (login, pagos, reembolsos).
- Validar que los mocks simulan correctamente el proveedor de pagos.
- Ejecutar pruebas manuales básicas en staging.
- Revisar logs y reportes generados por la IA.
- Confirmar permisos del workflow y del despliegue.
- Si todo está correcto, desplegar con feature flag o despliegue progresivo.

BLOQUE 5 — Investigación propia (cita la fuente)

▣ **Investiga:** Este bloque requiere que investigues por tu cuenta. Usa una fuente fiable y cita siempre de dónde lo sacaste (URL, nombre del artículo o documentación oficial).

5.1 — OWASP Top 10 y la IA

Hemos mencionado OWASP Top 10 como estándar de seguridad. Investiga dos categorías de OWASP Top 10 que no hayamos detallado en clase y explica cómo la IA podría detectarlas automáticamente en un pipeline CI/CD:

La IA puede revisar patrones de código y configuraciones igual que un auditor, pero más rápido.

Si ve algo como `hashlib.md5()` o un endpoint HTTP en vez de HTTPS, lo marca como riesgo.

La IA puede leer configuraciones igual que un desarrollador.

Si ve que un workflow tiene permisos de escritura innecesarios o que un contenedor usa root, lo marca como inseguro.

Campo	Tu respuesta
Categoría 1 (nombre + código OWASP)	A02:2021 - Cryptographic Failures Problemas relacionados con cifrado débil, datos sensibles sin proteger, uso incorrecto de HTTPS, etc. ¿Cómo lo detecta la IA en un pipeline? ➤ Analizando el código en busca de datos sensibles transmitidos sin cifrar. ➤ Detectando uso de algoritmos obsoletos (MD5, SHA1). ➤ Revisando configuraciones de HTTPS, certificados o claves demasiado cortas. ➤ Identificando si se guardan contraseñas sin hashing.
Categoría 2 (nombre + código OWASP)	A05:2021 - Security Misconfiguration Errores de configuración: permisos demasiado amplios, puertos expuestos, debug activado, etc. ¿Cómo lo detecta la IA en un pipeline? ➤ Revisando archivos YAML del pipeline (GitHub Actions, GitLab CI). ➤ Detectando permisos excesivos como write-all. ➤ Identificando configuraciones inseguras en Docker (ej: contenedores corriendo como root). ➤ Revisando si el servidor tiene el modo debug activado.
Fuente consultada (URL)	➤ OWASP Official Website - OWASP Top 10 https://owasp.org/Top10/ ➤ https://owasp.org/search/?searchString=+2021 ➤ https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/ ➤ https://owasp.org/Top10/2021/A02_2021-Cryptographic_Failures/
Fecha de consulta	26/03/2026

5.2 — Una herramienta de reportes automáticos de pipeline

Investiga una herramienta real que genere reportes automáticos para pipelines CI/CD (ejemplos: Allure Report, ReportPortal, TestRail, Codecov...). Describe qué información reporta, con qué plataformas es compatible y si integra IA:

Campo	Tu respuesta
Nombre de la herramienta	Allure Report
¿Integra IA?	No directamente, pero puede integrarse con herramientas de IA externas que analicen sus reportes. Allure no tiene IA nativa, pero se usa mucho junto a IA en pipelines modernos.
Compatible con (plataformas CI/CD)	➤ GitHub Actions ➤ GitLab CI ➤ Jenkins ➤ CircleCI ➤ Azure DevOps
¿Qué información reporta?	➤ Resultados de tests (pasados, fallados, ignorados). ➤ Tiempo de ejecución por test. ➤ Capturas de pantalla (en tests E2E). ➤ Logs y stacktrace de errores. ➤ Métricas de estabilidad del proyecto. ➤ Tendencias históricas de fallos.
Fuente (URL)	https://docs.qameta.io/allure/
Fecha de consulta	26/03/2026